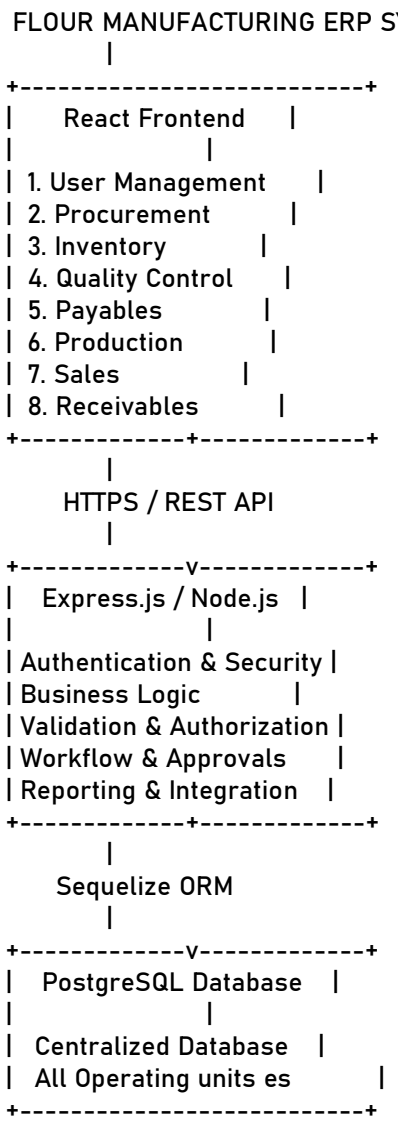
Flour Manufacturing ERP System: System Architecture, Technology Stack, Frontend and Backend Project Structure, and Business Process Integration

This structure will support your eight main modules, allow multiple developers to work independently, and make it easier to maintain and expand the system as your flour factory grows.

1. Recommended System Architecture

Your system should follow a centralized client-server architecture.



Recommended technology stack

Layer	Technology	Purpose
Frontend	React + Vite	ERP user interface
Programming language	JavaScript or TypeScript	Application development
Styling	Tailwind CSS	Responsive user interface
Routing	React Router	Navigation between modules
State management	Zustand	Shared frontend state
API communication	Axios	Communicate with the backend
Backend	Node.js + Express.js	Business logic and REST APIs
Database	PostgreSQL	Centralized data storage
ORM	Sequelize	Database models, queries and migrations
Authentication	JWT + secure password hashing	User authentication
Validation	Zod or Joi	Validate incoming data
Testing	Vitest, React Testing Library, Jest or Supertest	Frontend and backend testing
API documentation	Swagger / OpenAPI	Document and test REST APIs
Deployment	Linux server + Nginx	Host the application and route requests

For a new system expected to grow over time, TypeScript is worth considering for both frontend and backend because it provides stronger type checking. However, JavaScript with ES modules is also suitable if that is your preferred development approach.

2. React Frontend Folder Architecture

I recommend a **feature-based folder structure** rather than putting every page in a single components or pages folder.

Each business module should have its own pages, components, services, hooks, and validation rules. Shared components should be kept separately so all modules can reuse them.

Recommended project structure

```
flour-erp-frontend/
│
├── public/
│   ├── images/
│   ├── icons/
│   └── favicon.ico
│
├── src/
│   │
│   ├── app/
│   │   ├── App.jsx
│   │   ├── router.jsx
│   │   ├── providers.jsx
│   │   └── routeConfig.js
│   │
│   ├── assets/
│   │   ├── images/
│   │   ├── icons/
│   │   └── fonts/
│   │
│   ├── components/
│   │   ├── common/
│   │   │   ├── DataTable.jsx
│   │   │   ├── SearchInput.jsx
│   │   │   ├── DatePicker.jsx
│   │   │   ├── FileUpload.jsx
│   │   │   ├── StatusBadge.jsx
│   │   │   └── ConfirmDialog.jsx
│   │   │
│   │   ├── forms/
│   │   │   ├── FormInput.jsx
│   │   │   ├── FormSelect.jsx
│   │   │   ├── FormTextArea.jsx
│   │   │   └── FormActions.jsx
│   │   │
│   │   └── feedback/
│   │       ├── LoadingSpinner.jsx
│   │       ├── ErrorMessage.jsx
│   │       └── ToastNotification.jsx
│   │
│   ├── layouts/
│   │   ├── MainLayout.jsx
│   │   ├── AuthLayout.jsx
│   │   ├── Sidebar.jsx
│   │   ├── Topbar.jsx
│   │   ├── Breadcrumb.jsx
│   │   └── PageContainer.jsx
│   │
│   ├── features/
│   │   │
│   │   ├── auth/
│   │   │   ├── pages/
│   │   │   │   ├── LoginPage.jsx
```

```
│   │   │   │   └── ForgotPasswordPage.jsx
│   │   │   ├── components/
│   │   │   ├── services/
│   │   │   │   └── authService.js
│   │   │   └── hooks/
│   │   │
│   │   ├── user-management/
│   │   ├── procurement/
│   │   ├── inventory/
│   │   ├── quality-control/
│   │   ├── payables/
│   │   ├── production/
│   │   ├── sales/
│   │   └── receivables/
│   │
│   ├── services/
│   │   ├── apiClient.js
│   │   └── fileService.js
│   │
│   ├── hooks/
│   │   ├── useDebounce.js
│   │   └── usePagination.js
│   │
│   ├── store/
│   │   ├── authStore.js
│   │   ├── userStore.js
│   │   └── uiStore.js
│   │
│   ├── utils/
│   │   ├── dateUtils.js
│   │   ├── numberUtils.js
│   │   ├── currencyUtils.js
│   │   └── permissionUtils.js
│   │
│   ├── constants/
│   │   ├── apiEndpoints.js
│   │   ├── appConstants.js
│   │   └── statusConstants.js
│   │
│   ├── styles/
│   │   └── index.css
│   │
│   └── main.jsx
│
├── .env
├── .env.example
├── .gitignore
├── eslint.config.js
├── package.json
├── vite.config.js
└── README.md
```

The eight module folders should not remain empty. Each should contain the subfolders and files appropriate to its business functions.

3. Detailed Frontend Structure for Each Module

3.1 User Management

Responsibilities: users, roles, permissions, access control and user accounts.

```
features/user-management/
│
├── pages/
│   ├── UserListPage.jsx
│   ├── UserCreatePage.jsx
│   ├── UserEditPage.jsx
│   ├── RoleListPage.jsx
│   └── PermissionPage.jsx
│
├── components/
│   ├── UserForm.jsx
│   ├── UserTable.jsx
│   ├── RoleForm.jsx
│   └── PermissionMatrix.jsx
│
├── services/
│   └── userManagementService.js
│
├── hooks/
│   └── useUsers.js
│
├── schemas/
│   └── userSchema.js
│
└── utils/
    └── userHelpers.js
```

3.2 Procurement

Responsibilities: purchasing requests, approvals, purchase orders, supplier delivery records and goods-receiving coordination.

```
features/procurement/
│
├── pages/
│   ├── PurchaseRequestListPage.jsx
│   ├── PurchaseRequestCreatePage.jsx
│   ├── PurchaseRequestApprovalPage.jsx
│   ├── PurchaseOrderListPage.jsx
│   ├── PurchaseOrderCreatePage.jsx
│   ├── SupplierDeliveryListPage.jsx
│   └── SupplierDeliveryEntryPage.jsx
│
├── components/
│   ├── PurchaseRequestForm.jsx
│   ├── PurchaseOrderForm.jsx
│   ├── PurchaseOrderItems.jsx
│   ├── SupplierSelector.jsx
│   ├── DeliveryEntryForm.jsx
│   └── ProcurementApprovalPanel.jsx
│
├── services/
│   └── procurementService.js
│
├── hooks/
│   └── useProcurement.js
│
├── schemas/
│   └── procurementSchema.js
│
└── utils/
    └── procurementHelpers.js
```

Important: Procurement should manage purchasing and delivery documentation. Actual stock acceptance and posting to inventory should be controlled by the receiving and inventory workflows to prevent duplicate stock updates.

3.3 Inventory

Responsibilities: warehouses, stores, bins, raw materials, finished goods, stock movements, stock cards and stock adjustments.

```
features/inventory/
│
├── pages/
│   ├── ItemListPage.jsx
│   ├── ItemCreatePage.jsx
│   ├── WarehouseListPage.jsx
│   ├── StoreListPage.jsx
│   ├── BinManagementPage.jsx
│   ├── GoodsReceiptListPage.jsx
│   ├── GoodsReceiptCreatePage.jsx
│   ├── StockBalancePage.jsx
│   ├── StockCardPage.jsx
│   ├── BinCardPage.jsx
│   ├── StockTransferPage.jsx
│   ├── PhysicalCountPage.jsx
│   └── StockAdjustmentPage.jsx
│
├── components/
│   ├── ItemForm.jsx
│   ├── WarehouseForm.jsx
│   ├── GoodsReceiptForm.jsx
│   ├── StockMovementTable.jsx
│   ├── StockBalanceTable.jsx
│   ├── BinSelector.jsx
│   └── StockAdjustmentApproval.jsx
│
├── services/
│   └── inventoryService.js
│
├── hooks/
│   └── useInventory.js
│
├── schemas/
│   └── inventorySchema.js
│
└── utils/
    └── inventoryHelpers.js
```

For your flour factory, inventory should distinguish between:

- **Raw materials:** wheat, premix ingredients and packaging materials.
- **Work in progress:** materials currently being processed, where applicable.
- **Finished goods:** packaged flour and other finished products.
- **Warehouse and bin balances:** quantities available at specific storage locations.
- **Stock ledger:** the recorded history of inventory transactions.

3.4 Quality Control

Responsibilities: incoming wheat inspections, laboratory testing, finished-flour testing and non-conformance management.

```
features/quality-control/
│
├── pages/
│   ├── IncomingInspectionListPage.jsx
│   ├── IncomingInspectionCreatePage.jsx
│   ├── WheatTestPage.jsx
│   ├── FinishedProductTestListPage.jsx
│   ├── FinishedProductTestPage.jsx
│   ├── QualitySpecificationPage.jsx
│   ├── QualityApprovalPage.jsx
│   └── NonConformanceListPage.jsx
│
├── components/
│   ├── WheatTestForm.jsx
│   ├── FlourTestForm.jsx
│   ├── TestParameterTable.jsx
│   ├── AcceptanceCriteria.jsx
│   ├── QualityResultBadge.jsx
│   └── NonConformanceForm.jsx
│
├── services/
│   └── qualityControlService.js
│
├── hooks/
│   └── useQualityControl.js
│
├── schemas/
│   └── qualitySchema.js
│
└── utils/
    └── qualityCalculations.js
```

The interface should support the quality parameters relevant to your factory, including moisture, ash, wet gluten, particle size and other defined specifications.

Acceptance limits should be configurable and controlled by authorized personnel. The backend should validate test results and enforce applicable quality-release rules.

3.5 Payables

Responsibilities: supplier invoices, payment requests, payment approvals and payment execution.

```
features/payables/
│
├── pages/
│   ├── SupplierInvoiceListPage.jsx
│   ├── SupplierInvoiceCreatePage.jsx
│   ├── SupplierInvoiceDetailsPage.jsx
│   ├── PaymentRequestListPage.jsx
│   ├── PaymentRequestCreatePage.jsx
│   ├── PaymentApprovalPage.jsx
│   ├── PaymentExecutionPage.jsx
│   └── SupplierAccountPage.jsx
│
├── components/
│   ├── SupplierInvoiceForm.jsx
│   ├── InvoiceItemTable.jsx
│   ├── PaymentRequestForm.jsx
│   ├── PaymentExecutionForm.jsx
│   ├── PaymentAllocationTable.jsx
│   └── PaymentStatusBadge.jsx
│
├── services/
│   └── payablesService.js
│
├── hooks/
│   └── usePayables.js
│
├── schemas/
│   └── payablesSchema.js
│
└── utils/
    └── paymentCalculations.js
```

The frontend should distinguish payment requests, approved payments and executed payments. Approval must not automatically mean that money has been transferred.

3.6 Production

Responsibilities: production planning, milling operations, material consumption, production output and production quality.

```
features/production/
│
├── pages/
│   ├── ProductionPlanListPage.jsx
│   ├── ProductionPlanCreatePage.jsx
│   ├── ProductionOrderListPage.jsx
│   ├── ProductionOrderDetailsPage.jsx
│   ├── MillingBatchPage.jsx
│   ├── MaterialConsumptionPage.jsx
│   ├── ProductionOutputPage.jsx
│   ├── ProductionLossPage.jsx
│   ├── ProductionReportPage.jsx
│   └── ProductionCostPage.jsx
│
├── components/
│   ├── ProductionPlanForm.jsx
│   ├── ProductionOrderForm.jsx
│   ├── BatchDetailsForm.jsx
│   ├── MaterialConsumptionTable.jsx
│   ├── ProductionOutputTable.jsx
│   ├── YieldSummary.jsx
│   └── ProductionStatusBadge.jsx
│
├── services/
│   └── productionService.js
│
├── hooks/
│   └── useProduction.js
│
├── schemas/
│   └── productionSchema.js
│
└── utils/
    └── productionCalculations.js
```

Production must integrate with inventory so that material consumption and finished-product output update stock through authorized, validated backend transactions.

For example:

Raw Wheat → Milling → Flour and By-products → Quality Inspection → Finished-Goods Release

The system should also record milling losses and reconcile input quantities against output, losses and remaining work in progress.

3.7 Sales

Responsibilities: customer orders, pricing, dispatch, sales invoices and sales returns.

```
features/sales/
│
├── pages/
│   ├── CustomerListPage.jsx
│   ├── CustomerCreatePage.jsx
│   ├── SalesOrderListPage.jsx
│   ├── SalesOrderCreatePage.jsx
│   ├── SalesOrderDetailsPage.jsx
│   ├── DispatchListPage.jsx
│   ├── DispatchCreatePage.jsx
│   ├── SalesInvoiceListPage.jsx
│   └── SalesReturnPage.jsx
│
├── components/
│   ├── CustomerForm.jsx
│   ├── SalesOrderForm.jsx
│   ├── SalesOrderItems.jsx
│   ├── ProductPriceSelector.jsx
│   ├── DispatchForm.jsx
│   └── SalesInvoiceForm.jsx
│
├── services/
│   └── salesService.js
│
├── hooks/
│   └── useSales.js
│
├── schemas/
│   └── salesSchema.js
│
└── utils/
    └── salesCalculations.js
```

Sales should check product availability before dispatch and connect invoicing to receivables. Avoid allowing the same invoice to be posted independently in both modules.

3.8 Receivables

Responsibilities: customer balances, receipts, receipt allocations, outstanding invoices and collection reporting.

```
features/receivables/
│
├── pages/
│   ├── CustomerAccountPage.jsx
│   ├── CustomerInvoiceListPage.jsx
│   ├── CustomerReceiptListPage.jsx
│   ├── CustomerReceiptCreatePage.jsx
│   ├── ReceiptAllocationPage.jsx
│   ├── OutstandingInvoicePage.jsx
│   ├── CustomerStatementPage.jsx
│   └── ReceivablesAgingPage.jsx
│
├── components/
│   ├── CustomerReceiptForm.jsx
│   ├── ReceiptAllocationTable.jsx
│   ├── CustomerBalanceCard.jsx
│   ├── OutstandingInvoiceTable.jsx
│   └── AgingSummary.jsx
│
├── services/
│   └── receivablesService.js
│
├── hooks/
│   └── useReceivables.js
│
├── schemas/
│   └── receivablesSchema.js
│
└── utils/
    └── receivablesCalculations.js
```

Receivables should track customer payments and allocate receipts against outstanding invoices. Partial payments, unapplied receipts, credit balances and overdue invoices should be handled according to your accounting rules.

4. Express.js / Node.js Backend Folder Architecture

For the backend, I recommend a **modular monolith architecture**.

This means the eight business modules run within one backend application but maintain separate business logic, routes, controllers, services, models and validation rules.

It is simpler to develop and deploy than microservices, while providing a clear path for future expansion.

4.1 Complete Backend Project Structure

```
flour-erp-backend/
│
├── src/
│   │
│   ├── server.js
│   ├── app.js
│   │
│   ├── config/
│   │   ├── database.js
│   │   ├── environment.js
│   │   ├── cors.js
│   │   └── logger.js
│   │
│   ├── database/
│   │   ├── migrations/
│   │   ├── seeders/
│   │   ├── factories/
│   │   └── scripts/
│   │
│   ├── middleware/
│   │   ├── authenticate.js
│   │   ├── authorize.js
│   │   ├── validateRequest.js
│   │   ├── errorHandler.js
│   │   ├── notFoundHandler.js
│   │   ├── requestLogger.js
│   │   └── uploadMiddleware.js
│   │
│   ├── modules/
│   │   │
│   │   ├── auth/
│   │   ├── user-management/
│   │   ├── procurement/
│   │   ├── inventory/
│   │   ├── quality-control/
│   │   ├── payables/
```

```
│   │   ├── production/
│   │   ├── sales/
│   │   └── receivables/
│   │
│   ├── shared/
│   │   ├── constants/
│   │   ├── enums/
│   │   ├── errors/
│   │   ├── helpers/
│   │   ├── validators/
│   │   ├── utilities/
│   │   └── types/
│   │
│   ├── integrations/
│   │   ├── email/
│   │   ├── sms/
│   │   ├── file-storage/
│   │   └── accounting/
│   │
│   ├── jobs/
│   │   ├── scheduledJobs.js
│   │   └── reportJobs.js
│   │
│   └── tests/
│       ├── integration/
│       ├── unit/
│       └── fixtures/
│
├── storage/
│   └── uploads/
│
├── logs/
│
├── docs/
│   ├── api/
│   └── system-design/
│
├── .env
├── .env.example
├── .gitignore
├── eslint.config.js
├── package.json
└── README.md
```

Note: The `storage/uploads/` and `logs/` directories are deployment-dependent. In production, uploaded documents are often better stored in protected external or object storage, while logs should be managed using an appropriate logging system.

4.2 Standard Backend Structure for Each Module

Each module should follow a consistent structure.

For example, the inventory module:

```
modules/inventory/
│
├── routes/
│   └── inventory.routes.js
│
├── controllers/
│   └── inventory.controller.js
│
├── services/
│   └── inventory.service.js
│
├── models/
│   ├── Item.js
│   ├── Warehouse.js
│   ├── Store.js
│   ├── Bin.js
│   ├── GoodsReceipt.js
│   ├── GoodsReceiptItem.js
│   ├── StockLedger.js
│   └── StockBalance.js
│
├── repositories/
│   └── inventory.repository.js
│
├── validators/
│   └── inventory.validator.js
│
├── policies/
│   └── inventory.policy.js
│
├── constants/
│   └── inventory.constants.js
│
└── tests/
    ├── inventory.service.test.js
    └── inventory.integration.test.js
```

The exact files will vary by module. For example, procurement will have purchase-request and purchase-order models, while quality control will have inspection, test-result and quality-specification models.

What each backend folder does

Folder	Responsibility
routes/	Defines API endpoints and connects them to controllers.
controllers/	Receives HTTP requests, invokes services and returns responses.
services/	Implements business rules, workflows and transaction coordination.
models/	Defines Sequelize models and database relationships.
repositories/	Encapsulates database queries when a separate data-access layer is useful.
validators/	Validates incoming request data.
policies/	Enforces module-specific access and business authorization rules.
constants/	Defines module-specific constants and permitted values.
tests/	Contains automated unit and integration tests.

Important design principle: Controllers should not contain complex SQL or business rules. Keep business logic in services and database operations in models or repositories. Use Sequelize transactions for operations that must succeed or fail together.

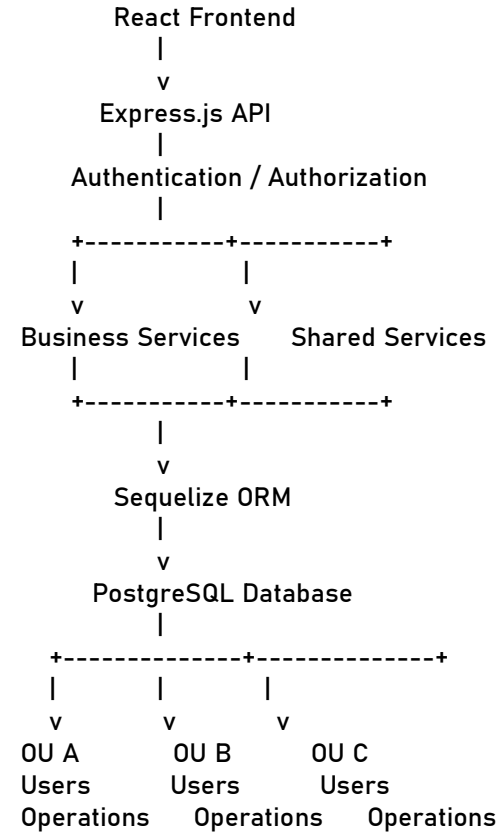
4.3 Backend Structure for All Eight Modules

Module	Main backend responsibilities	Example models
User Management	Users, roles, permissions and access control	User, Role, Permission, UserRole
Procurement	Purchase requests, approvals and purchase orders	PurchaseRequest, PurchaseOrder, PurchaseOrderItem
Inventory	Warehouses, bins, receipts and stock movements	Item, Warehouse, Bin, GoodsReceipt, StockLedger
Quality Control	Incoming inspections, laboratory tests and quality release	Inspection, QualityTest, TestResult, QualitySpecification
Payables	Supplier invoices, payment requests and payment execution	SupplierInvoice, PaymentRequest, SupplierPayment
Production	Production plans, milling batches and material consumption	ProductionOrder, ProductionBatch, MaterialConsumption, ProductionOutput
Sales	Customer orders, dispatches and sales invoices	SalesOrder, SalesOrderItem, Dispatch, SalesInvoice
Receivables	Customer receipts, allocations and outstanding balances	CustomerReceipt, ReceiptAllocation, CustomerAccount

These are representative model names, not a complete database schema. Each model will need appropriate fields, relationships, constraints and audit information.

5. Recommended Database Architecture

For your flour manufacturing system, I recommend **one centralized PostgreSQL database** for all operating units es, with a shared data model and controlled access.



The operating units access the same application and database. Operating units - specific records should be associated with an appropriate operating units_id or organizational identifier, where relevant.

However, not every table needs a operating units _id. For example, centrally managed item definitions may be shared across operating units es, while stock balances and warehouse transactions must identify their relevant locations.

6. How the Modules Should Communicate

The modules should not operate as isolated applications. They should follow connected business workflows.

A. Procurement to Inventory to Payables

```
Purchase Request
    |
    v
Approval
    |
    v
Purchase Order
    |
    v
Supplier Delivery
    |
    v
Incoming Quality Inspection
    |
    +---- Failed ----> Rejection / Return Workflow
    |
    v
Goods Receipt
    |
    v
Inventory Stock Posting
    |
    v
Supplier Invoice
    |
    v
Payment Request
    |
    v
Payment Approval
    |
    v
Payment Execution
```

The backend should enforce the relevant prerequisites. For example, a supplier invoice should be checked against the purchase order and goods receipt where required by your business rules.

B. Inventory to Production to Quality Control

Available Raw Wheat
|
v
Production Order
|
v
Material Issue
|
v
Milling Process
|
v
Production Output
|
v
Finished Flour Quality Testing
|
+---- Failed ----> Quality Hold / Non-Conformance
|
v
Quality Release
|
v
Finished Goods Inventory

Only authorized, quality-released finished goods should be available for dispatch when your quality policy requires release before sale.

C. Sales to Inventory to Receivables

```
Customer Order
    |
    v
Stock Availability Check
    |
    v
Order Approval (if required)
    |
    v
Dispatch
    |
    v
Sales Invoice
    |
    v
Customer Receivable
    |
    v
Customer Receipt
    |
    v
Receipt Allocation
    |
    v
Updated Outstanding Balance
```

A posted sales invoice should be recorded once and referenced by the receivables module. This avoids creating duplicate invoices or receivable balances.

7. Recommended API Structure

Use a consistent REST API prefix, such as /api/v1.

Module	Example endpoint	Purpose
Authentication	POST /api/v1/auth/login	Authenticate a user
User Management	GET /api/v1/users	List authorized users
Procurement	POST /api/v1/procurement/purchase-orders	Create a purchase order
Inventory	GET /api/v1/inventory/stock-balances	Retrieve stock balances
Quality Control	POST /api/v1/quality-control/inspections	Record an inspection
Payables	POST /api/v1/payables/payment-requests	Create a payment request
Production	POST /api/v1/production/orders	Create a production order
Sales	POST /api/v1/sales/orders	Create a sales order
Receivables	POST /api/v1/receivables/receipts	Record a customer receipt

Additional endpoints should support approvals, cancellations, posting, reporting and other operations as required.

For example, approving a purchase order should normally be an explicit business operation, rather than simply allowing the frontend to update an `approval_status` field directly.

8. Recommended Development Order

Although all eight modules are part of the overall system, their implementation should follow their dependencies and be organized around complete, end-to-end business processes. The development should cover the entire raw material procurement-to-payment process, the production process, and the sales-to-receivables process, ensuring seamless integration and data consistency across all modules.

9. Final Recommendations

For your project, I would use the following conventions:

- **Frontend:** React + Vite + Tailwind CSS + React Router + Zustand.
- **Backend:** Node.js + Express.js using ES modules.
- **Database:** PostgreSQL with Sequelize migrations.
- **Architecture:** Feature-based frontend and modular monolith backend.
- **API:** Versioned REST endpoints.
- **Security:** Authentication, role-based permissions, input validation and audit logging.
- **Deployment:** One centrally managed application and database, accessible to authorized users across all operating units es.
- **Testing:** Automated unit, integration and end-to-end tests.
- **Documentation:** Maintain API documentation and database relationship diagrams.

One additional recommendation: Treat inventory and financial posting as controlled business processes. A purchase order, goods receipt, supplier payment, production output or customer receipt should not change posted balances simply because a user clicks Save. The backend should validate the operation, check permissions, apply the appropriate transaction and preserve an auditable record.

This architecture provides a solid foundation for a maintainable flour manufacturing ERP system without introducing the complexity of microservices too early.